



香港大學

THE UNIVERSITY OF HONG KONG

AIF-HKU Capstone Project

The Fabrication of Tactile Sensors and the Construction of Test Environment

Name: Liu Zhaolin

Date: 18/01/2026

CONTENTS



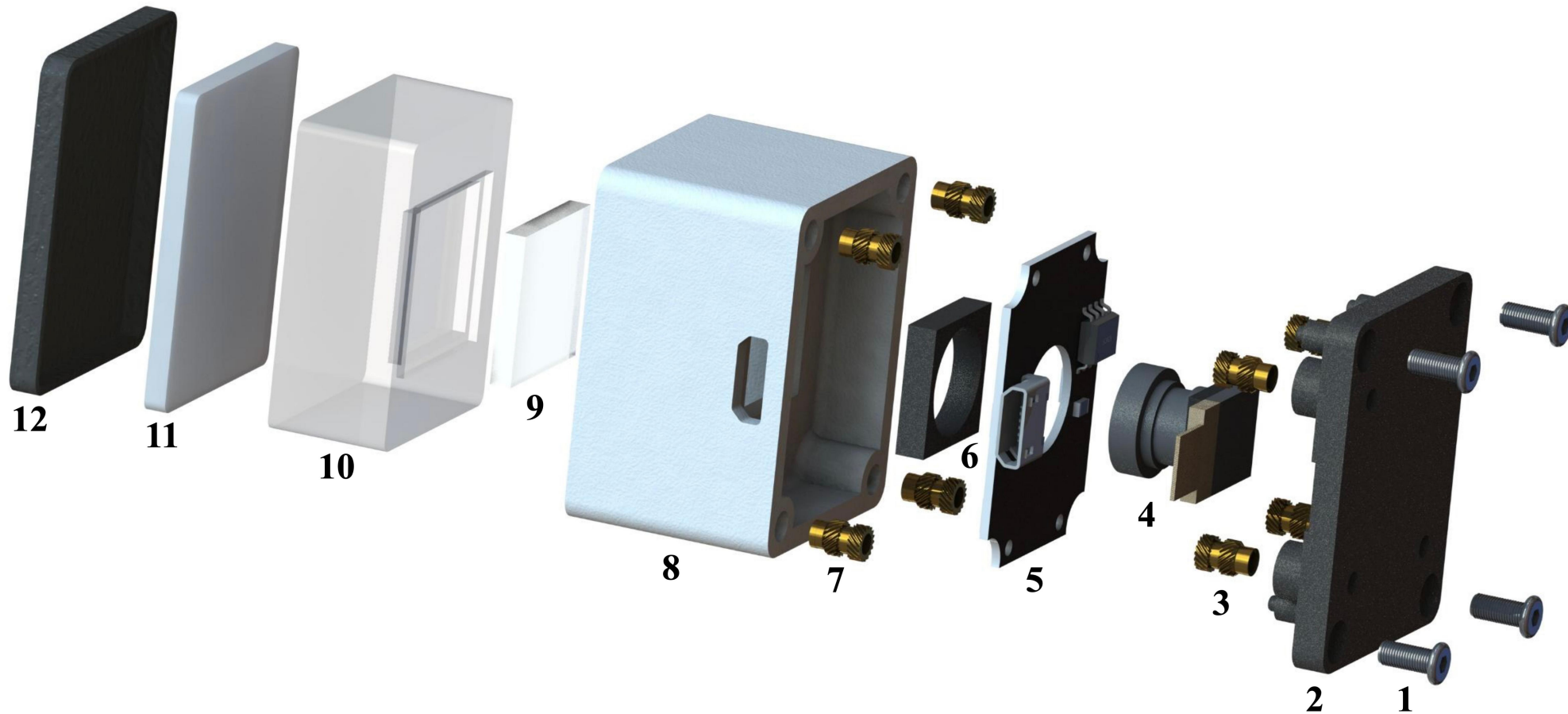
- 1. Fabrication of Tactile Sensors (9DTact)**
- 2. Construction of Test Environment & Pressure-Sensitive Image Demo**



01.

Fabrication of Tactile Sensor

1.1 The Structure of Tactile Sensor



1.2 Physical Photo of the Sensor



non-operating state



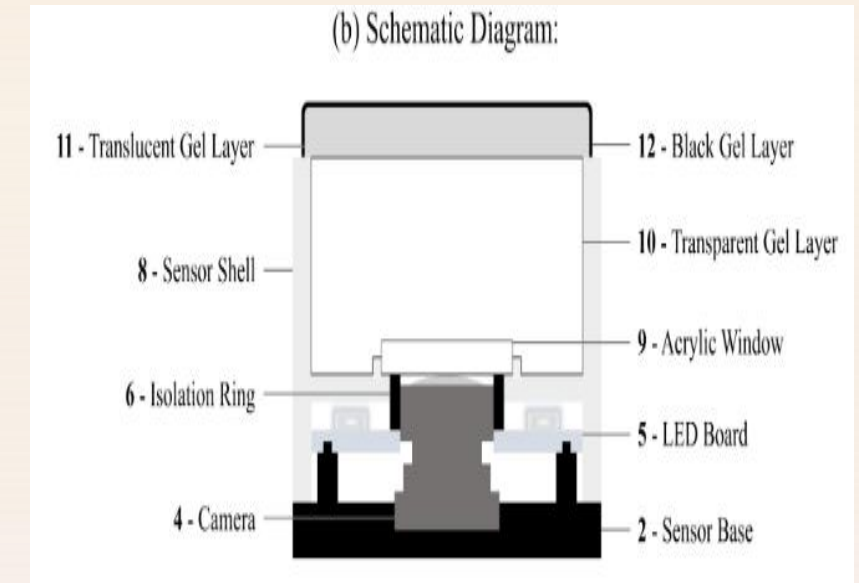
operating state

1.3 Fabrication Difficulties

(1) Coating Uniformity: The reflective layer on the silicone must be applied with extreme precision to prevent optical signal distortion.

(2) Bubble Control: Vacuum degassing is mandatory during silicone casting to eliminate air pockets that cause image artifacts.

(3) Light Source: LEDs require precise equidistant soldering to ensure a perfectly uniform internal light field for 3D reconstruction.



02.

Construction of Test Environment & Pressure-Sensitive Image Demo

2.1 The Process of Construction & Test

Display copy: third-party
robot photo omitted.

Our sensor adopts the
OV5647 camera module,
which requires
calibration first.



The entire environment is set
up on the Ubuntu 20.04
system, Python Version: 3.8
PyTorch Version: 1.10
ROS Version: ROS1

Create a Pressure-
Sensitive Image

2.2 Some Relevant Code

```

10 class CameraCalibration:
11     def calibrate_image(self, ref, sample):
12         position_scale.append(position_per_img)
13         print(position_scale)
14         np.save(self.camera.position_scale_path, position_scale)
15
16         # draw the points
17         for point in all_point:
18             cv2.circle(sample_drawing, (point[1], point[0]), point_size, command_color, -1)
19         # draw the central point with special color
20         for around_index in (-self.col_points, -1, 0, 1, self.col_points):
21             cv2.circle(sample_drawing, (all_point[center_index + around_index][1],
22                                     all_point[center_index + around_index][0]), point_size + 4,
23                                     special_point_color, -1)
24         cv2.imshow('sample_drawing', sample_drawing)
25
26         # calculate the real position for the sampling points
27         init_position = all_point.copy()
28         real_position = np.zeros_like(init_position)
29         for i in range(self.row_points):
30             for j in range(self.col_points):
31                 real_position[i*self.col_points+j] = init_position[center_index] + \
32                     dis_avg * np.array([i - self.row_points // 2, j - self.col_points // 2])
33
34         # get the row and column indexes for correcting distortion
35         itp_row = Rbf(real_position[:, 0], real_position[:, 1], init_position[:, 0], function='cubic')
36         itp_col = Rbf(real_position[:, 0], real_position[:, 1], init_position[:, 1], function='cubic')
37         col_mesh, row_mesh = np.meshgrid(range(ref.shape[1]), range(ref.shape[0]))
38         row_index = itp_row(row_mesh, col_mesh).astype(np.int32)
39         col_index = itp_col(row_mesh, col_mesh).astype(np.int32)
40
41         # make sure no outside index
42         for i in range(ref.shape[0]):
43             for j in range(ref.shape[1]):
44                 if row_index[i, j] < 0 or row_index[i, j] > 479 or col_index[i, j] < 0 or col_index[i, j] > 639:
45                     row_index[i, j] = 0

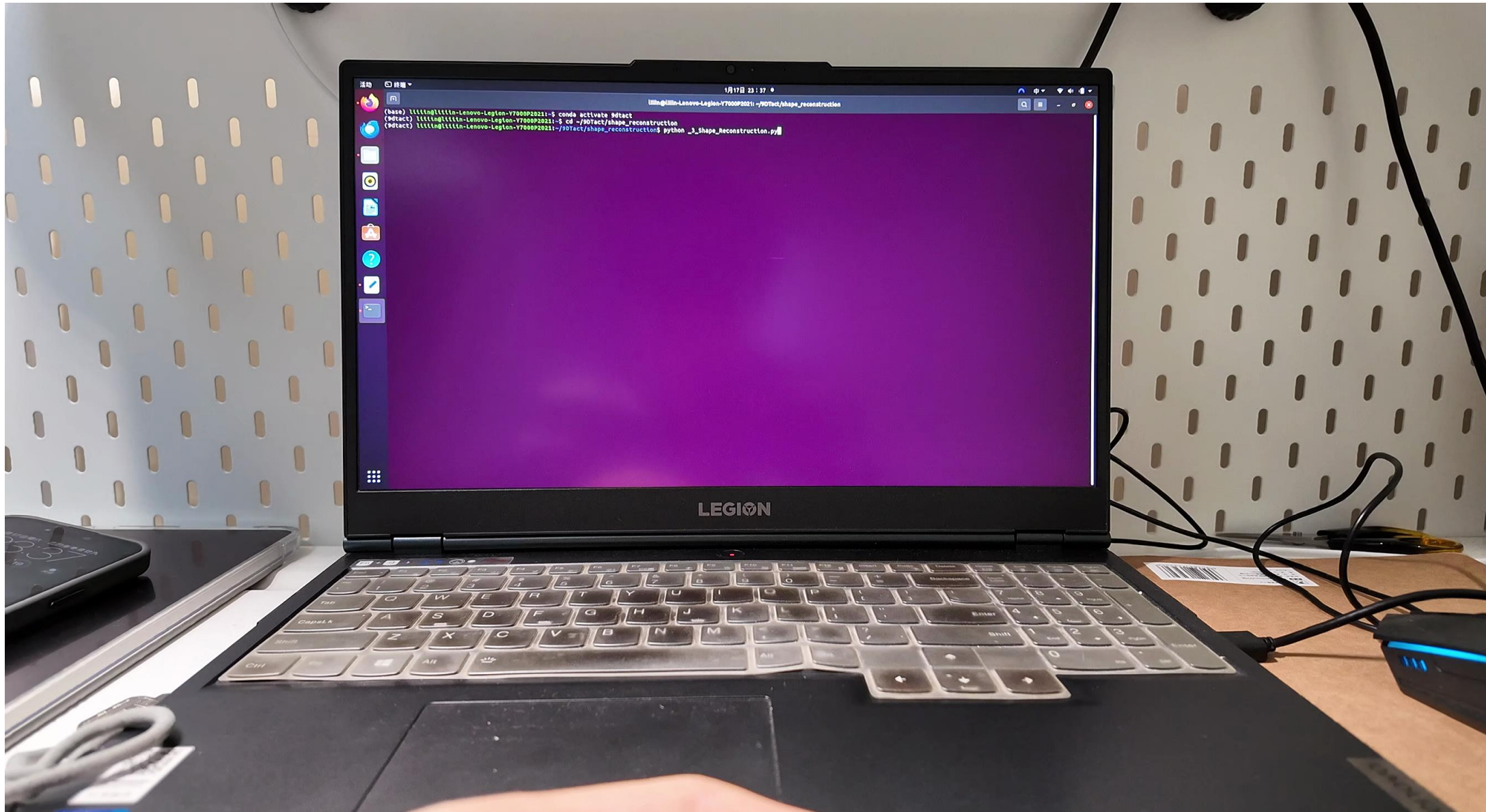
```

```

1 # collect images and calibrate the sensor
2 import numpy as np
3 import cv2
4 import yaml
5 import os
6 from shape_reconstruction import Sensor
7
8
9 class SensorCalibration:
10     def __init__(self, cfg_path):
11         f = open(cfg_path, 'r+', encoding='utf-8')
12         cfg = yaml.load(f, Loader=yaml.FullLoader)
13         self.sensor = Sensor(cfg, calibrated=False)
14         if not os.path.exists(self.sensor.depth_calibration_dir):
15             os.makedirs(self.sensor.depth_calibration_dir)
16         depth_calibration = cfg['depth_calibration']
17         self.BallRad = depth_calibration['BallRad']
18         self.circle_detection_gray = depth_calibration['circle_detect_gray']
19         self.show_circle_detection = depth_calibration['show_circle_detection']
20
21     def run(self):
22         print("DON'T touch the sensor surface!!!!")
23         print('Please press "y" to save the reference image!')
24         ref = self.sensor.get_raw_avg_image()
25         cv2.imwrite(self.sensor.depth_calibration_dir + '/ref.png', ref)
26         rc_ref = self.sensor.rectify_crop_image(ref)
27         cv2.imwrite(self.sensor.depth_calibration_dir + '/rectify_crop_ref.png', rc_ref)
28         print('Reference image saved!')
29         print('Please press the ball on the sensor and press "y" to save the sample image!')
30         sample = self.sensor.get_raw_avg_image()
31         cv2.imwrite(self.sensor.depth_calibration_dir + '/sample.png', sample)
32         rc_sample = self.sensor.rectify_crop_image(sample)
33         cv2.imwrite(self.sensor.depth_calibration_dir + '/rectify_crop_sample.png', rc_sample)
34         print('Sample image saved!')
35

```

2.3 Video of Testing Process





香 港 大 學

THE UNIVERSITY OF HONG KONG

Thanks!

Name: Liu Zhaolin

Date: 18/01/2026